BYOL ACADEMY

The code, explained

A concise, beginner-friendly walkthrough of the actual website source: how pages are assembled, how the demo works, how enquiries are protected, and how to find the code you want to change.

> What this PDF is
> A companion to BYOL_Academy_Beginner_Guide.pdf. The earlier guide includes website screenshots and the broader project walkthrough; this one concentrates on source code and the meaning of its important patterns.

Written for a beginner. You can read the excerpts without copying them. The full implementation lives in the named TypeScript/TSX files.

First: how a browser gets a page

A visitor enters a URL. Next.js maps that URL to a route file under src/app. That file returns React components (the page structure). Components read content objects. CSS classes style the output. Some components also run browser-side JavaScript for interaction.

```
URL: /
   |
   v
src/app/page.tsx  -> composes homepage sections
   |                 uses shared content and components
   +--> src/content/site.ts
   +--> src/components/AdaptiveDemo.tsx
   +--> src/styles/globals.css
   |
   v
browser displays HTML, CSS and interactive behavior
```

Three terms that explain most of the code

Term	Plain meaning	Example
Component	Reusable piece of interface.	Header, Footer, AdaptiveDemo
Props	Values passed into a component.	MasteryMap receives estimates
State	A value the browser remembers as someone interacts.	Demo's current question and selected answer
Type	A contract describing what a value may contain.	Platform requires a name, link and approval flag

Server vs browser code
Most Next.js page files are rendered on the server by default. A file marked “use client” is allowed to use React state, browser events and browser storage. Keep client code for interactions that need it.

1. The shared frame and home page

Start with layout.tsx, then page.tsx. One provides shared furniture; the other decides the homepage order.

layout.tsx: the wrapper around every route

```
export default function RootLayout({ children }) {
  return (
    <html lang="en-IN">
      <body>
        <a className="skip-link" href="#main">Skip to content</a>
        <Header />
        <main id="main">{children}</main>
        <Footer />
        <AnalyticsEvents />
      </body>
    </html>
  );
}
```

Code	Meaning
lang="en-IN"	Sets document language for browsers and assistive technology.
children	The route page inserted inside this shared wrapper.
Header / Footer	Reusable navigation shown on every route.
skip-link	Keyboard users can skip repeated navigation.
AnalyticsEvents	Optional interaction tracking; event sending is gated by settings.

The same file exports metadata: default title, description, canonical base, social-preview metadata and preview no-index behavior. Individual pages can replace the title/description.

page.tsx: composing the homepage

```
export default function HomePage() {
  const visiblePlatforms = shippable(platforms);
  const visibleStats = shippable(stats);
  return (
    <>
      <JsonLd data={organizationJsonLd} />
      <section className="hero-section">...</section>
      <section id="audiences">...</section>
      <section id="platforms">...</section>
      <section id="how-it-works">...</section>
      <section id="see-it-adapt"><AdaptiveDemo /></section>
      <section id="responsible-ai">...</section>
    </>
  );
}
```

The page is a server component: it can create the page without browser state. It imports icons, reusable components and content, then arranges sections. “...” represents page markup omitted here, not a missing implementation.

Why arrays and map() appear often

```
platforms.map((platform) => (
  <article key={platform.id}>
    <h3>{platform.name}</h3>
    <p>{platform.summary}</p>
    <a href={platform.href}>{platform.ctaLabel}</a>
  </article>
))
```

map() creates one card for each platform object. `key` is a stable identifier React uses to keep the correct card when a list changes. The data stays separate from the repeated layout.

2. Content, types and approval flags

Text and product details are data, kept apart from the JSX that lays them out.

site.ts is the main content source

```
export type Platform = {
  id: "medicoplasma" | "schools" | "colleges";
  name: string;
  audience: string;
  summary: string;
  features: string[];
  href: string;
  ctaLabel: string;
  approved: boolean;
};
```

This TypeScript type describes a product card. A string is text; string[] is a list of text; the id union permits only the three written IDs; approved must be true or false. The compiler can flag a product missing required information.

```
export const brand = {
  name: "BYOL Academy",
  siteUrl: process.env.NEXT_PUBLIC_SITE_URL
    ?? "https://byolacademy.com",
} as const;
```

`export` allows other files to import this object. `process.env` reads configuration. `??` uses the right-hand fallback only when the setting is null/undefined. `as const` preserves literal values and makes the object read-only to TypeScript.

Approval filtering prevents accidental publication

```
export function shippable<T extends { approved: boolean }>(
  items: readonly T[],
): T[] {
  return process.env.NODE_ENV === "production"
    ? items.filter((item) => item.approved)
    : [...items];
}
```

This generic helper accepts any list whose items have an approved flag. In production it returns only approved entries; in development it returns a copy of the list so the team can preview drafts. Stats and platforms use this path.

What belongs where?

Change	File
Website copy, brand, products, navigation, stats	src/content/site.ts
Demo sample questions	src/content/demo-data.ts
Page section order and layout	src/app/page.tsx
Colors, spacing, responsive rules	src/styles/globals.css
Form fields and their constraints	src/components/LeadForm.tsx and src/lib/validation/lead.ts

3. Interactive demo: state to result

AdaptiveDemo.tsx is the main client-side feature. It tracks a small finite flow rather than showing every screen at once.

The state variables

```
type DemoState = "idle" | "asking" | "feedback" | "results";
const [state, setState] = useState<DemoState>("idle");
const [questionIndex, setQuestionIndex] = useState(0);
const [selected, setSelected] = useState<number | null>(null);
const [estimates, setEstimates] = useState<number[]>([]);
const [correctCount, setCorrectCount] = useState(0);
```

Value	What it remembers
state	Which screen is shown.
questionIndex	Current question, starting at zero.
selected	Chosen answer number, or null if unanswered.
estimates	One illustrative probability per answered question.
correctCount	Number of correct answers so far.

Submitting an answer

```
function submitAnswer() {
  if (selected === null || !question) return;
  const correct = selected === question.answer;
  const estimate = updateMastery(initialEstimate, correct);
  setEstimates((previous) => [...previous, estimate]);
  setCorrectCount((previous) => previous + Number(correct));
  setState("feedback");
}
```

The guard exits if an answer/question is missing. The selected index is compared with the correct answer index. The helper calculates a sample estimate. The functional setState form uses the previous value, so it safely appends/increments even if updates are queued. Finally the feedback view appears.

Moving forward and restarting

moveForward() checks whether the current question is the last one. If yes, it changes state to results; otherwise it advances the index, clears the selected answer and returns to asking. reset() clears the same values back to the initial screen.

Updating an estimate

```
const likelihood = correct
  ? prior * (1 - slip)
  : prior * slip;
const notMasteredLikelihood = correct
  ? (1 - prior) * guess
  : (1 - prior) * (1 - guess);
const evidence = likelihood + notMasteredLikelihood;
const posterior = evidence === 0 ? prior : likelihood / evidence;
return posterior + (1 - posterior) * learn;
```

`prior` is the old estimate from 0 to 1. The model parameters allow for guessing, slipping and learning. Bayes-style evidence produces a posterior estimate; the last line adds a chance of learning. Inputs are checked and invalid probabilities throw a RangeError.

> Important
> This BKT-style example is a local teaching illustration only. It is not BYOL's deployed model, a validated score, or a real learner record.

Accessibility and the map

The demo uses labelled radio inputs and fieldsets. It moves focus to the new question/results heading after screen changes. MasteryMap draws an SVG, but also renders a semantic HTML table with equivalent values for assistive technology. The visualization is therefore not the only way to get the information.

4. Contact page and server API

The form has two halves: a visitor interface and a server endpoint. The server must validate again because browser inputs can be changed.

Client form: collect and report

```
const response = await fetch("/api/lead", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload),
});
const result = await response.json();
if (!response.ok || !result.ok) {
  setStatus(result.message ?? "Please try again later.");
  return;
}
setStatus("Thank you. Your enquiry has been sent.");
```

fetch() sends the form to the same website. JSON.stringify converts the object into JSON. The response status and `ok` flag decide whether to show an error or success. A try/catch handles network failures. At present, a public feature flag keeps the form hidden and shows why it is unavailable.

Zod schema: define accepted data

```
export const leadSchema = z.object({
  name: z.string().trim().min(2).max(100),
  email: z.string().trim().email().max(200),
  intent: z.enum(["learner", "parent", "institution", "other"]),
  consent: z.literal(true),
}).refine(
  (value) => value.intent !== "institution" ||
    Boolean(value.organisation && value.organisation.length > 0),
  { path: ["organisation"], message: "Please tell us your institution." },
);
```

A schema is a runtime contract. `trim()` removes outer spaces; min/max bound lengths; email checks the address shape; enum limits intent to known options; literal(true) requires explicit consent. refine() adds a cross-field rule for institutions.

Server route: order of checks

Step in route.ts	What it protects
Content type + Origin	Accept expected JSON from this site only.
10,000-byte streamed body cap	Reject oversized requests before buffering them all.
JSON parse + object check	Reject malformed JSON and non-object input.
Honeypot	Drop simple bot submissions.
leadSchema.safeParse()	Validate untrusted fields and return field errors.
Server flags + credentials	Do not accept/deliver until legal approval and providers are configured.
Hashed IP + Upstash limit	Limit repeated traffic; hash the IP before using it as a key.
Resend email	Deliver a plain-text message using private server credentials.

Why the form is disabled
Privacy/consent language needs legal approval, and production email and rate-limit services need credentials. Both the visible form and server route are separately gated; setting only a browser flag is not enough.

5. Shared components, styling and search

Smaller files each take one responsibility. This makes behavior easier to find and maintain.

File	Responsibility
Header.tsx	Mobile-menu state, keyboard Escape handling, navigation and demo CTA.
AnnouncementBar.tsx	Dismissible announcement stored for the current tab session.
Footer.tsx	Reusable product/company/legal links.
MasteryMap.tsx	SVG concept graph, status colors and equivalent hidden table.
JsonLd.tsx + lib/seo.ts	Structured search descriptions for Organization and Website.
AnalyticsEvents.tsx + lib/analytics.ts	Optional data-attribute click and scroll events.
ApprovalNotice.tsx	Development-only indicator for unapproved copy.

CSS: design tokens first

```
:root {
  --color-ink: #0b1b3b;
  --color-canvas: #faf7f0;
  --color-primary: #1f3a8a;
  --color-accent: #f5b301;
  --radius-card: 16px;
}
.shell {
  width: min(100% - 48px, 1200px);
  margin-inline: auto;
}
```

CSS custom properties beginning with `--` are reusable design values. `var(--color-ink)` reads one. `.shell` is a shared maximum-width container. The stylesheet also holds component classes, breakpoints for tablet/phone layouts, keyboard focus and reduced-motion behavior.

Search support and security settings

File	Purpose
layout.tsx	Titles, description, canonical URL, social metadata.
robots.ts	Crawler access; preview sites can be blocked.
sitemap.ts	List of public pages for crawlers.
opengraph-image.tsx	Social-sharing preview image.
next.config.ts	Security response headers and powered-by header setting.

These settings make pages easier to describe and reduce some common browser risks. They do not guarantee search rankings or replace a security review.

6. Tests, configuration and safe changes

Tests are executable examples of expected behavior. Configuration files tell tools how to run the project.

File/group	What it checks/configures
tests/unit/mastery.test.ts	Correct answers raise estimates, incorrect answers lower them, invalid inputs fail.
tests/unit/content.test.ts	Approval filtering and demo data expectations.
tests/unit/lead-validation.test.ts	Accepted and rejected lead values.
tests/unit/lead-route.test.ts	API origin, size, bot and configuration safeguards.
tests/e2e/site.spec.ts	Real pages, demo interactions, mobile menu, horizontal overflow and axe accessibility.
package.json	Commands and exact package versions.
tsconfig.json + eslint.config.mjs	Type checking and lint rules.
vitest.config.ts + playwright.config.ts	Unit-test runner and mobile/desktop browsers.
.env.example	Variable names only; never real passwords or tokens.

Useful commands

```
npm install          # install locked dependencies
npm run dev          # local editing server
npm run lint         # code quality rules
npm run typecheck    # TypeScript checks only
npm test             # unit tests
npm run test:e2e     # browser tests
npm run build        # production build
npm run start        # serve built site
```

A safe beginner workflow

Step	Action
1	Open a file using Ctrl+P in VS Code.
2	For wording/links, start in src/content/site.ts.
3	For layout, inspect the page/component that renders it.
4	Make one related change; do not edit node_modules.
5	Run lint, typecheck and the relevant tests.
6	Run the site locally and inspect desktop and phone widths.
7	Ask the content/legal owner to approve factual or legal text before launch.

Before enabling real enquiries
Obtain approved privacy/consent text and configure the private Resend and Upstash settings in the hosting provider. Never commit credentials.

Quick index: where to edit

If you want to change…	Open…
Homepage sections	src/app/page.tsx
Words or product destinations	src/content/site.ts
Demo question text	src/content/demo-data.ts
Demo estimate mathematics	src/lib/adaptive/mastery.ts
Contact UI	src/components/LeadForm.tsx
Contact server checks	src/app/api/lead/route.ts
Color/layout/mobile rules	src/styles/globals.css

End of code guide - For screenshots and the broader beginner project walkthrough, see BYOL_Academy_Beginner_Guide.pdf.